

CS 4361

Assignment 1: Hello Teapot! Introduction to Three.js, WebGL, and Shaders

Due 11:59PM, September 21, 2026

1 Introduction

The main goals of this assignment are to setup your graphics development environment, including checking your browser compatibility, setting up a local server, and an initial exploration of the uses of vertex and fragment shaders. For this exploration, you will be using a template provided by the instructor, including shader code (`.glsl` files in the `glsl/` folder). Your main work will be to develop a high-level understanding of how the code works, to modify or write shaders, and to use basic communication between the JavaScript program and the shaders. Some of the details of what is going on in the rest of the code will only become clear a bit later in the course. You are of course welcome to take a peek now, especially for the last part of the assignment. Some of the concepts are explained in the web resources.

To program a shader, you will use a programming language called GLSL (OpenGL ES Shading Language version 3.0). Note that there are several versions of GLSL, with more advanced features, available in regular OpenGL. Make sure that any code you find while trying to learn GLSL is the correct version.

This assignment uses a simple scene consisting of a “Teapot” character and a magical “Orb” that it interacts with. You can move the camera around the scene by dragging with a mouse, pan by holding down the right mouse button while dragging, and zoom by scrolling the mouse wheel. Your task for this assignment will be to write simple shaders to move the Orb around, turn it on to illuminate the teapot, detect how close it is to the teapot, and make it interact with the teapot’s body.

1.1 Getting the Code

The assignment code is released on the e-learning page. Please download it to your local machine, navigate to the folder where you intend to keep your assignment code, and run the following command from the terminal or command line:

```
unzip a1-release.zip
```

1.2 Template

- The file `A1.html` is the launcher of the assignment. Open it in your preferred browser to run the assignment, to get started.
- The file `A1.js` contains the JavaScript code used to set up the scene and the rendering environment. You will need to make minor changes in it to answer the questions.
- The folder `glsl` contains the vertex and fragment shaders for the teapot and lightbulb geometry. This is where you will do most of your coding.
- The folder `js` contains the required JavaScript libraries. You do not need to change anything here.
- The folder `obj` contains the geometric models loaded in the scene.
- The folder `images` contains the texture images used.

1.3 Execution

As mentioned above, the assignment can be run by opening the file `A1.html` in any modern browser. However, most browsers will prevent pages from accessing local files on your computer. If you simply open `A1.html`, you may get a black screen and an error message on the console similar to this:

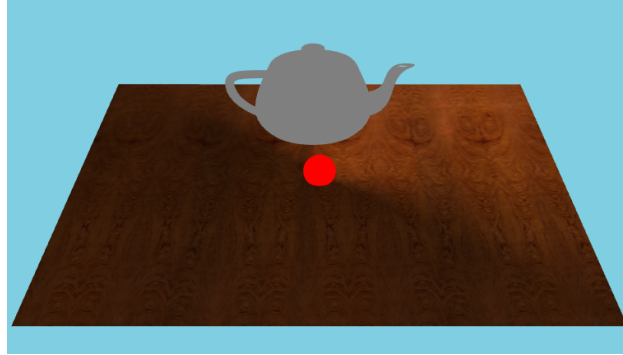
```
XMLHttpRequest cannot load... Cross origin requests are only
supported for protocol schemes: http, data, https.
```

We highly recommend that you run a local server, instead of changing browser security settings. For example you can do this with VS Code:

1. Follow the link <https://code.visualstudio.com/Download> to download and install VS Code.
2. Open VS Code and install the Live Server extension. You may also install it from here: <https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer>
3. In VS Code, open the assignment's root folder.
4. Open `A1.html`, right-click in the editor, click "Go Live" in the right down corner.

2 Work to be done (100 pts)

First, ensure that you can run the template code in your browser. See the instructions above. Study the template to get a sense of how it works. The script `js/setup.js` creates the basic scene with the floor, and provides a utility function for loading 3D models. The initial configuration should look as it does in the figure below.

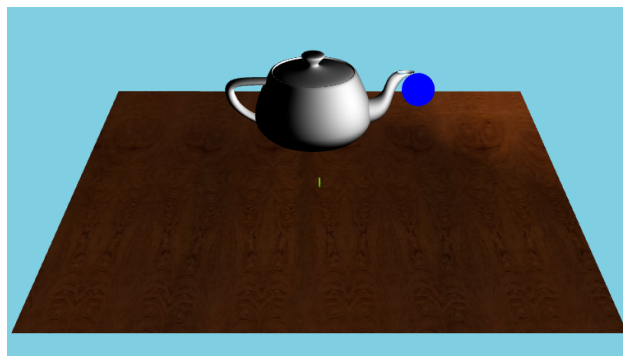


Part 1: Required Elements

- (a) **20 pts** Moving & Coloring the Orb. The shape of the Orb is represented by `SphereGeometry`, and manipulated using the vertex shader `sphere.vs.glsl`. The variable `orbPosition` (the position of the orb center in world coordinates) is declared in `A1.js`. It is changed using the keyboard, and passed to the sphere vertex shader using a `uniform` variable. First, modify the sphere shader to move the sphere in response to keyboard input. Then, change the color of the sphere to blue in the sphere fragment shader (in `sphere.fs.glsl`). Important: **do not** use Three.js functions; you must modify the shader for credit.

- (b) **20 pts** Lighting the Teapot.

The light from the orb should light up the teapot. Here you will implement a simple model of how light from the orb would interact with the teapot, a simple shading model called “Gouraud shading.” We will study more realistic models later in the course. Modify `A1.js` and `teapot.vs.glsl` to color each vertex of the teapot based on the cosine of the angle between its normal and the direction vector to the center of the sphere. When correctly coded, the orb will be “activated”, lighting up different parts of the teapot as it’s moved around, as illustrated in the figure below.



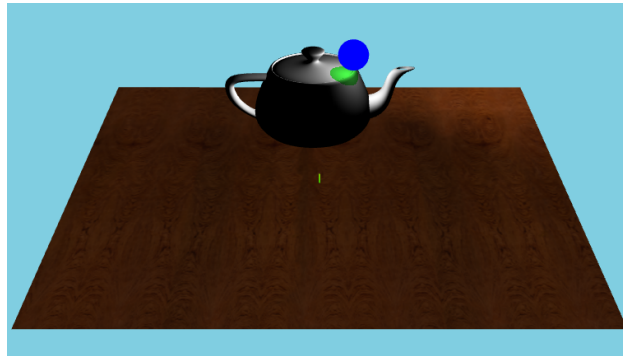
Hint 1: See how uniforms are passed to the sphere shader.

Hint 2: You should pass the necessary information about the sphere to the teapot shaders.

Hint 3: See how varying variables are passed to the teapot fragment shader.

(c) **30 pts** Proximity detection.

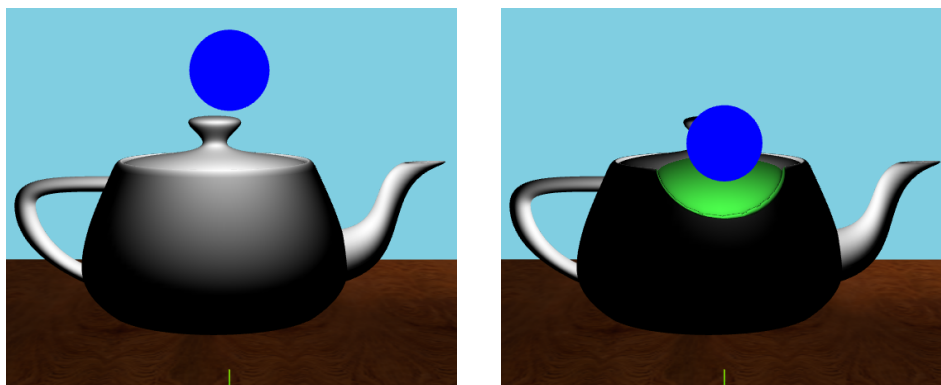
The teapot has sensors on its skin that can detect objects in close proximity. For this part you will need to modify `teapot.fs.glsl` and `teapot.vs.glsl` to further color the teapot fragments green when in close proximity to the sphere, as illustrated in the figures below. One simple way is to check if a teapot fragment is within a specified distance to the sphere, and if it is, set its color to green.



Hint: You should use the appropriate uniform variable in the teapot shader.

(d) **30 pts** Body Deformation.

In this part you will indent the teapot's mesh when pushed in by the Orb, as illustrated in the figure below. This is a preview of how vertex shaders can be used for changing a shape. For this you will need to change `teapot.vs.glsl` and `A1.js`. One simple way is to check if a vertex is within the Orb, and if it is, move the vertex to the surface. You should pass the necessary information about the Orb to the teapot shader.



Hint 1: See how uniforms are passed on the sphere shaders.

Part 2: Creative License (Optional)

You have many opportunities to unleash your creativity in computer graphics! In this **optional** section, and you are invited to extend the assignment in fun and creative ways. We'll highlight some of the best work in class. A small number of exceptional contributions may be awarded bonus points. Some possible suggestions might be:

- explode the teapot or orb along face normals.
- animate colors, lights, in fun ways.
- add interesting objects to the scene.

3 Submission Instructions

3.1 Directory Structure

You must submit your final code and write a clear **README** file which includes your name, student number, and the core idea/changed code/explanation/screenshot of each question. The TAs strongly encourage you to read the template of answers in `template.md` and use a similar template to submit your answers.

Your **README** file can be in PDF, Word, or Markdown format. It must be placed in the root directory and will serve as the basis for grading your work. **Missing a README file will result in a 30-point deduction.**

3.2 Submission Methods

Please compress everything under the root directory of your assignment into `a1.zip` and submit it on E-learning. You can make multiple submissions, but we will grade only the last one.

4 Grading

4.1 Point Allocation

Each assignment has 100 points for Part 1. Part 2 is optional, and you can get bonus points (0-10 points) at the instruction team's discretion. The max score for each assignment is 110 points. **Missing a README file will result in a 30-point deduction.**

4.2 Grading

The TA will carefully review your **README** file and evaluate based on the idea/code/explanation/screenshot. If the program shows correct rendering effects, full points will be graded.

4.3 Anti-AI/Plagiarism Measures

To prevent plagiarism, 12 students will be randomly selected for face-to-face grading. These students are required to present their final submitted code to the instructor during office hours and point out the locations of core code for the corresponding problems.

4.4 Penalties

Aside from penalties for incorrect solution or plagiarism, we may apply the following penalties to each assignment:

Late penalty. A deduction of 10 points would be applied to each extra late day. Note that

- (a) We check the time of your last submission to determine if you are late or not;
- (b) We do not consider Part 1 and Part 2 submissions separately. Say if you submitted Part 1 on time but updated your submission for Part 2 one day after the deadline, that counts as one late day.

AI/Plagiarism penalty. If a student cannot point out where their code is located for solving specific problems (e.g., which lines of GLSL code correspond to the answer for a particular problem) and cannot clearly articulate their approach to solving the problem, this will result in a complete deduction of points for the corresponding assignment.